Fault Analysis of Microscaling Formats on a RISC-V SoC

Dillibabu Shanmugam
Worcester Polytechnic Institute
Worcester, MA, USA
dshanmugam@wpi.edu

Patrick Schaumont
Worcester Polytechnic Institute
Worcester, MA, USA
pschaumont@wpi.edu

Abstract

Microscaling (MX) formats share one exponent across an element block, creating a new fault surface: a single-bit flip in the shared exponent corrupts all block elements simultaneously. Five MX-compatible 8-bit encodings (MXINT8, MXFP8-E4M3, MXFP8-E5M2, LOG8-SUM, and LOG8-MAX) are compared through exhaustive single-bit weight fault injection across four workloads using gate-level simulation on the CAPRI1 RISC-V SoC, with the primary workload additionally validated on fabricated silicon. Format choice alone can substantially change vulnerability. Three MX-specific mechanisms explain this variation: block-shared exponent amplification, attention routing inversion from exponent-field faults, and log-domain fault filtering in the max-reduction variant. No single format dominates all axes: LOG8-MAX leads in speed and energy but is limited by accuracy on some attention workloads, MXINT8 provides the strongest fault resilience with full accuracy, and LOG8-SUM offers a balanced compromise across speed, accuracy, and resilience. MX element encoding should be treated not only as an accuracy-efficiency choice, but also as a security-relevant design decision.

CCS Concepts

• **Security and privacy** → **Embedded systems security**; **Hardware security implementation**; • **Hardware** → *Fault tolerance*; • **Computing methodologies** → *Neural networks.*

Keywords

Microscaling formats, fault injection, RISC-V, hardware security, log-domain arithmetic, attention mechanism

ACM Reference Format:
Dillibabu Shanmugam and Patrick Schaumont. 2026. Fault Analysis of Microscaling Formats on a RISC-V SoC. In *Great Lakes Symposium on VLSI 2026 (GLSVLSI '26), June 22–24, 2026, Canandaigua, NY, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3787109.3815291

1 Introduction

Edge AI systems increasingly adopt low-precision numeric formats to reduce memory, bandwidth, and energy. Microscaling (MX) formats [1] share a single scale value across an element block, improving storage efficiency while preserving dynamic range. Figure 1 introduces five MX-compatible 8-bit formats, including the

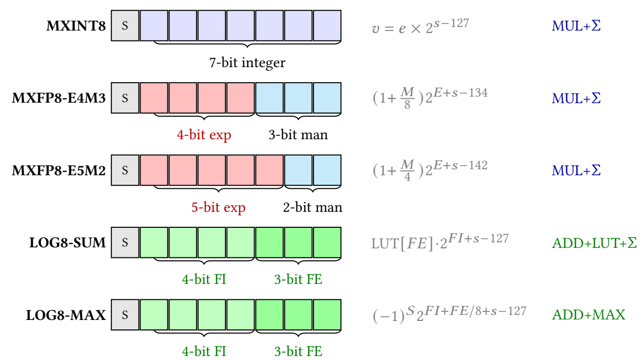

Figure 1: Five MX element encodings: bit-field layouts and decode formulas. MXFP8-E4M3's 4-bit element exponent (red) creates exponential fault sensitivity (2^1–2^8 per bit flip). LOG8-SUM and LOG8-MAX share the same [S:1][FI:4][FE:3] encoding; LOG8-SUM converts to linear via LUT before summation, while LOG8-MAX replaces MUL+Σ with ADD+MAX, bounding fractional-bit error to $\leq 1.84\times$. MXINT8 has purely linear error. The shared exponent s (E8M0) applies to all five formats.

LOG8 encoding of Dally [7] evaluated here in two reduction variants (LOG8-SUM and our max-reduction LOG8-MAX), each with distinct trade-offs for coefficient storage and inference speed. All share the E8M0 block scale; they differ in element interpretation and reduction arithmetic. As MX moves from standardization toward deployment, its evaluation must extend beyond accuracy and efficiency to include fault resilience.

Fault resilience matters most in embedded and safety-critical systems, where transient disturbances or targeted fault injection can corrupt stored weights and silently alter model outputs. A bit flip in a conventional per-element representation perturbs one value. A bit flip in an MX shared exponent perturbs an entire block simultaneously. This block-level coupling means MX formats exhibit fault behavior qualitatively distinct from standard INT8 or FP32 representations.

Prior MX hardware work [8–10] addresses throughput, area, and accuracy, while neural-network fault studies [2, 3, 15] target conventional formats. A comparative study examining fault behavior across MX encodings on a common platform remains absent. This gap matters because MX formats differ structurally in ways that affect fault propagation, as we detail in Section 2.

We study five MX-compatible 8-bit formats through exhaustive gate-level single-bit SRAM fault injection on the CAPRI1 netlist, derived from a fabricated TSMC 180 nm RISC-V SoC with scan-chain observability. This setup provides controlled format-to-format comparison under a single-bit model while preserving the timing and state visibility of the target platform. It supports both misclassification analysis and fault propagation tracking through scan-chain state capture.

Our results show that format choice materially changes vulnerability. No single format dominates all axes: LOG8-MAX leads in

GLSVLSI '26, Canandaigua, NY, USA

ACM ISBN 979-8-4007-2431-2/2026/06
https://doi.org/10.1145/3787109.3815291

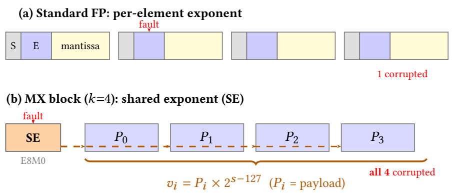

Figure 2: Standard FP vs. MX block. (a) Per-element exponent; a fault corrupts one value. (b) MX shares one exponent (SE) across k=4 elements; a single SE bit flip shifts *all* values by $2^{\pm\Delta}$ (P_i = element payload).

speed (2–4.6×) and energy, MXINT8 provides the strongest fault resilience with full accuracy, and LOG8-SUM balances all three without hardware multipliers. MXFP8-E4M3 is the most vulnerable format, while LOG8-MAX achieves zero misclassifications on the primary workload (Exp 1) but is limited to 50% golden accuracy on one multi-token attention workload. These results suggest that MX format selection is a hardware-security design decision, not merely an accuracy–efficiency trade-off. Artifacts: https://github.com/Secure-Embedded-Systems/capri1_mx_faultanalysis.

The contributions are as follows:

(1) A **five-format comparison across four workloads** on the CAPRI1 platform (TSMC 180 nm RISC-V SoC), with the primary workload additionally validated on fabricated silicon, using exhaustive fault analysis and scan-chain observability.
(2) A **mechanistic explanation** of an up-to-8× vulnerability disparity on the primary workload (0%–8.9%), tracing it to exponent bit-width, reduction path, and attention routing sensitivity.
(3) Evidence that **no single format dominates all axes**: LOG8-MAX leads in speed/energy but caps at ∼50% golden accuracy on multi-token attention; MXINT8 leads in resilience with full accuracy; LOG8-SUM is Pareto-balanced.

Section 2 introduces the five MX formats and their hardware cost. Section 3 defines the threat model and formalizes security properties. Section 4 analyzes how each format responds to faults within the attention dataflow. Section 5 describes the CAPRI1 platform and workloads. Section 6 presents results from cross-workload overview to per-format mechanisms. Section 7 discusses implications and related work, and Section 8 concludes.

2 Background and Preliminaries

2.1 Microscaling Formats

An MX block [1] packs one shared scale byte (E8M0) with k element bytes (k=4 here), as shown in Figure 2. The decoded value $v_i = P_i \cdot 2^{s-127}$ couples every element to a single scale s. This coupling underlies both the compression benefit and the fault risk: a bit flip in the shared exponent rescales all k values at once, whereas a flip in an element byte affects only one.

2.2 Five MX-Compatible Encodings

Figure 1 shows the five 8-bit encodings evaluated in this work; all share the E8M0 block scale but differ in element interpretation and reduction arithmetic. **MXINT8** encodes each element as a signed 7-bit integer with no element exponent; a bit flip produces a bounded, linear perturbation proportional to the flipped bit position. **MXFP8-E4M3** adds a 4-bit element exponent atop the 8-bit shared exponent (12 effective exponent bits) and a 3-bit mantissa; a single flip can shift magnitude by 2^1 to 2^8. **MXFP8-E5M2** widens the element exponent to 5 bits with a 2-bit mantissa, trading granularity for dynamic range. **LOG8-SUM** uses the [S:1][FI:4][FE:3] log encoding proposed by Dally [7], where FI is a 4-bit integer log and FE is a 3-bit fractional log; each product is computed as an addition in the log domain, converted to linear via a lookup table, and then accumulated through summation. **LOG8-MAX** shares the same encoding as LOG8-SUM but replaces summation with max-reduction in the log domain, eliminating both the LUT conversion and the multiplier.

Table 1: Per-MAC cost on rv32ic. LOG8-MAX needs only 2 cycles (ADD+CMP); multiply-based formats require ∼20-cycle software multiply.

Format	Multiply	Decode	Accumulate	Cyc/MAC
LOG8-MAX	ADD (1)	–	CMP (1)	∼2
LOG8-SUM	ADD (1)	LUT+SHIFT (3)	ADD (1)	∼5
MXINT8	smul (20)	–	ADD (1)	∼21
MXFP8	smul (20)	decode (5)	ADD (1)	∼26

2.3 Log-Domain Arithmetic and Hardware Cost

Both log formats replace multiplication with a 7-bit integer addition: $p_i = \ell_{w_i} + \ell_{x_i}$. They diverge at reduction:

$$z_j^{\text{LOG8-SUM}} = \textstyle\sum_i \text{lin}(\ell_{w_i} + \ell_{x_i}) \quad \text{(convert, then sum)} \qquad (1)$$

$$z_j^{\text{LOG8-MAX}} = \max_i (\ell_{w_i} + \ell_{x_i}) \quad \text{(max, no conversion)} \qquad (2)$$

Table 1 compares per-MAC cost on the CAPRI1 rv32ic core: LOG8-MAX costs ∼2 cyc/MAC versus ∼21 for MXINT8 (2–4.6× speedup). At 180 nm [11] a 7-bit add (0.03 pJ) versus a 32-bit multiply (3.1 pJ) gives LOG8-MAX an estimated ∼8× per-MAC energy advantage.

2.4 Fault Observability

No standard mechanism observes a fault at the instant it corrupts internal state during inference. CAPRI1 (Figure 6) integrates a 12,756-flip-flop scan chain (originally for manufacturing test) that captures the complete processor state every 50 cycles; the Hamming distance between golden and faulted snapshots turns a binary pass/fail outcome into time-resolved propagation analysis.

3 Threat Model and Security Analysis

3.1 Threat Model

The adversary targets SRAM-resident model weights for *silent misclassification* (the program completes but returns the wrong class). Among physical attack mechanisms (EMFI, voltage glitching, power-supply manipulation), we adopt a single-bit fault model that isolates format-dependent behavior from multi-bit masking effects and represents a profiled, calibrated attacker. We assume white-box knowledge of weight byte addresses and exclude instruction corruption, control-flow hijacking, and denial-of-service faults.

3.2 SE and Element Faults

Attention is increasingly common in edge AI inference [16], and each weight matrix in the attention dataflow stores MX blocks of SE and element bytes; Figure 3 shows the fault paths.

Under the single-bit fault model, a weight fault targets either the shared exponent or an element byte. An SE fault (Level 1, Figure 4(a))

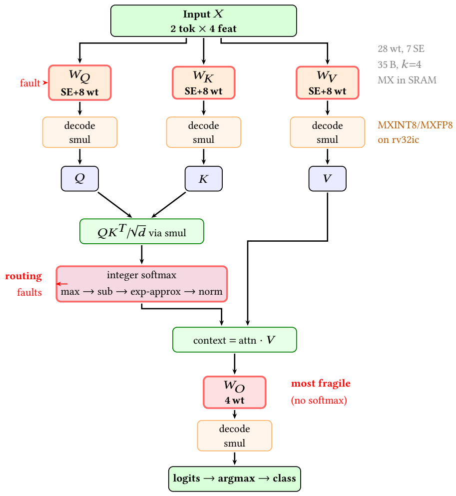

Figure 3: Attention inference on CAPRI1 rv32ic for MXINT8/MXFP8 formats. Each weight matrix stores MX blocks (SE + k=4 elements) in SRAM. LOG8 formats replace decode+smul with log-domain ADD (Table 1). W_O maps to logits without softmax (most fragile).

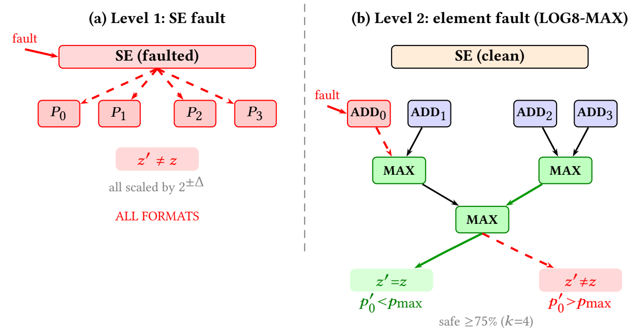

Figure 4: Two-level fault behavior. (a) SE fault shifts all k=4 elements by $2^{\pm\Delta}$; all formats are equally vulnerable. (b) Element fault in LOG8-MAX: MAX discards faults below the maximum (Δz=0); summation-based formats always propagate.

shifts all k block values by $2^{\pm\Delta}$; all five formats are equally vulnerable at this level. An element fault (Level 2, Figure 4(b)) corrupts one decoded value. In summation-based formats, every corrupted term shifts the accumulated output. In LOG8-MAX, the max operator discards the corrupted term if its value remains below the current maximum.

3.3 Security Properties

The following properties formalize the containment guarantees that follow from this two-level structure.

Definition 1 (Classification Margin). *Let $\hat{y} = \arg\max_j z_j$ be the predicted class. The classification margin $m = z_{\hat{y}} - \max_{j \neq \hat{y}} z_j \geq 0$ is the gap between the two largest signed logits; it is non-negative by construction (no absolute value is needed because $\hat{y}$ is defined as the argmax, so $z_{\hat{y}}$ is the largest logit regardless of its sign). A fault is exploitable* iff *it changes $\hat{y}$.*

Fault amplification. Summation-based formats propagate every faulted term to the output. Worst-case amplification: MXINT8 254×, LOG8-SUM 256× (integer-part flip at b=3: 2^8), MXFP8-E5M2 65,536× (2^{16} from a 5-bit element-exponent bit flip). For LOG8-MAX, the fault on term k produces:

$$\Delta z_j^{\text{LD}} = \begin{cases} 0 & \text{if } (\ell_{w'_k} + \ell_{x_k}) \leq M_{-k} \\ (\ell_{w'_k} + \ell_{x_k}) - M_{-k} & \text{otherwise} \end{cases} \quad (3)$$

where $M_{-k} = \max_{i \neq k}(\ell_{w_i} + \ell_{x_i})$. The perturbation is exactly zero when the faulted term does not become the new maximum.

Property 1 (Local Containment). *Let $p^* = \max_i p_i$ and $g_k = p^* - p_k$. A single-bit fault on term k is absorbed whenever $\delta_b \leq g_k$. Under uniform targeting, the containment rate is at least $(n-1)/n$.*

Property 2 (Dual-Condition Requirement). *A fault changes the predicted class only when* (C1) *it propagates through the max-reduction* and (C2) *the resulting logit shift exceeds the classification margin m.*

These are structural observations under the single-bit model, not general security guarantees. Section 6 validates these bounds experimentally.

4 Format-Dependent Fault Behavior

4.1 Fault Sensitivity of MX Encodings

As Figure 2 shows, an SE fault corrupts all k elements, while an element fault corrupts one value. The impact of an element fault depends on how the format decodes magnitude and how it accumulates partial products.

Magnitude sensitivity. A bit flip in MXINT8 shifts the decoded value by an integer step proportional to the bit position, so the error grows linearly. MXFP8-E4M3 has 12 effective exponent bits (4 element + 8 shared); a single flip can shift magnitude by 2^1 to 2^8, so the error grows exponentially. MXFP8-E5M2 widens the element exponent to 5 bits and reduces the mantissa to 2 bits; this extends the range but makes each mantissa bit more influential under faults. LOG8-SUM and LOG8-MAX encode values in log space; the worst-case fractional-bit flip produces ≤ 1.84× error ($2^{7/8} \approx 1.84$), bounded by the 3-bit fractional field.

Reduction path. MXINT8, MXFP8-E4M3, MXFP8-E5M2, and LOG8-SUM accumulate through summation, so every corrupted term contributes to the final output. LOG8-MAX replaces summation with max-reduction, where only the largest term determines the output; a faulted term that remains below the maximum therefore has no effect on the result.

4.2 Shared-Exponent Amplification

As Figure 4(a) illustrates, an SE bit flip rescales all k block values by $2^{\pm\Delta}$ at once. This k× fan-out distinguishes MX from per-element formats, where a fault perturbs only one weight. The worst-case amplification depends on how each format decodes the shared exponent. MXFP8-E5M2 compounds the SE shift with its 5-bit element exponent, reaching up to 65,536× (the widest exponent range gives the largest decoded shift per flip). MXFP8-E4M3 reaches a smaller 256× from a single element-exp flip but is empirically the most vulnerable format on Exp 1 (Table 4) because its 3-bit mantissa increases the density of value-altering bit positions. LOG8-SUM amplifies up to 256× when a flip hits the integer part of the log field. MXINT8 limits amplification to 254× because its linear decode contains no element exponent. LOG8-MAX has the same SE

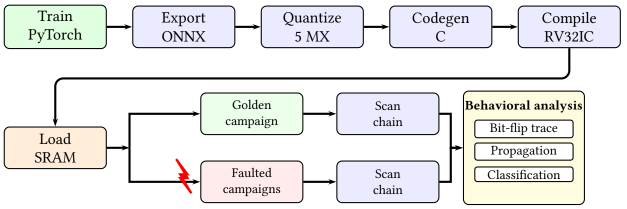

Figure 5: End-to-end fault-injection pipeline, common to simulation and CAPRI1 silicon; scan-chain captures feed behavioral analysis.

vulnerability as LOG8-SUM, but its max-reduction filters the error at the element level (Section 4.4).

4.3 Attention Dataflow and Fault Paths

The attention mechanism [6] projects input X through weight matrices W_Q, W_K, W_V and computes:

$$Q = XW_Q,\ K = XW_K,\ V = XW_V \tag{4}$$

$$\mathrm{Attn} = \mathrm{softmax}(QK^T/\sqrt{d_k})\,V \tag{5}$$

Figure 3 shows how CAPRI1 executes this dataflow with MX-encoded weights stored in SRAM. The softmax output determines which tokens contribute to each position. A fault in W_Q or W_K corrupts QK^T and distorts this distribution. In MXFP8-E4M3, a single exponent-field flip can shift a key magnitude by 2× to 64×; this is large enough to invert the softmax output, causing the model to route Token 0's information to Token 1's position and vice versa. We call this failure mode *attention routing inversion*. The output projection W_O maps context directly to logits without any normalizing layer, so faults in W_O pass to the classifier without attenuation.

4.4 Log-Domain Fault Filtering

In LOG8-MAX, the max-reduction operator selects only the largest term from each dot product, as Figure 4(b) illustrates. A fault on a non-winning term does not change the output because the corrupted value remains below the current maximum. For k=4, at least 75% of element faults target non-winners and produce no effect on the result. A misclassification therefore requires two conditions: the faulted term must overtake the winner, *and* the resulting logit shift must exceed the classification margin. Summation-based formats lack both conditions because every corrupted term shifts the accumulated output and propagates to the classifier.

5 Experiments

5.1 Experimental Flow

Figure 5 shows the end-to-end fault injection pipeline for evaluating all five MX formats on the CAPRI1 platform. The pipeline begins by training a model in PyTorch and exporting it to ONNX. The ONNX model is then quantized into five MX encodings from *identical* FP32 values (k=4). Format-specific C implementations are cross-compiled to RV32IC binaries and loaded into SRAM, placing the weight block at known addresses. Each fault campaign flips one weight bit before execution, and a matching golden run provides the baseline. The 12,756-FF scan chain captures the complete processor state every 50 cycles during the golden run and the faulted run. Hamming distance between these two snapshots measures fault propagation over time. This behavioral analysis traces each bit flip

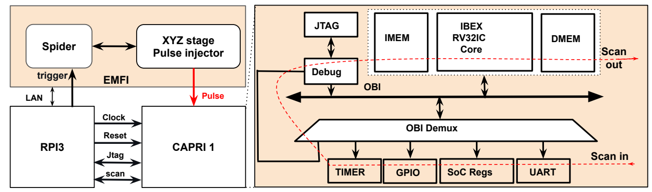

Figure 6: CAPRI1 SoC: Ibex RV32IC core with 1 KB DFF SRAM holding 35-byte MX weights at known DMEM addresses, plus a 12,756-FF scan chain that captures state every 50 cycles.

Table 2: Tiny Attention: 280 faults/format (35 B × 8 bits), simulation vs. CAPRI1 EMFI.

Format	Misclass.		Rate (%)	
	SIM	CAPRI1	SIM	CAPRI1
MXINT8	3/280	1/280	1.1	0.4
MXFP8-E4M3	**25/280**	**9/280**	**8.9**	**3.2**
MXFP8-E5M2	6/280	5/280	2.1	1.8
LOG8-SUM	3/280	1/280	1.1	0.4
LOG8-MAX	**0/280**	1/280	**0.0**	0.4

from initial corruption through intermediate state disturbance to final classification impact.

This pipeline executes on two platforms. *Gate-level simulation* with SDF timing runs all four workloads and provides deterministic single-bit control over every weight byte. The *fabricated CAPRI1 chip*, shown in Figure 6, validates the primary workload on silicon and confirms simulation predictions. CAPRI1 is a TSMC 180 nm RISC-V SoC (2×2 mm^2, Ibex CV32E20, RV32IC, 1.8 V) with 1 KB DFF-based SRAM and a 12,756-FF scan chain. EMFI uses a coil probe 1 mm above the die on a motorized XYZ stage, delivering ±20 V pulses of 4–8 ns width [17]; each format undergoes 280 EMFI campaigns (35 bytes × 8 bits). Table 2 compares simulation and silicon: the gate-level count is an upper bound under a deterministic single-bit model, while EMFI hits the targeted cell with probability <1 and may flip neighbors (multi-bit events excluded), biasing silicon counts downward.

5.2 Workloads

The primary workload is a single-head attention binary classifier sized to fit CAPRI1's 1 KB memory while preserving every stage relevant to fault analysis: projection (W_Q, W_K, W_V, 4×2 each), softmax routing, value aggregation, and output projection (W_O, 2×2). Totals: 28 weights, 35 packed bytes per format. All five encodings are quantized from identical FP32 values, ensuring any resilience difference is attributable solely to format choice. The architecture and fault paths follow Figure 3; Table 2 reports simulation versus CAPRI1 EMFI counts.

Three additional workloads extend coverage via gate-level simulation: a MoE attention classifier (28 weights), an MLP anomaly detector (114 parameters), and a keyword-spotting attention model (KWS, 106 parameters, 4 tokens). All are quantized identically to the primary workload. Table 3 reports cycle counts and available fault data across all four.

5.3 Metrics

Three metrics quantify each format's fault behavior. *Misclassification rate* counts silent failures where the faulted output differs from the golden output (Tables 2 and 3). *Cycle count* measures execution cost per format per workload (Table 3). *Scan-chain Hamming*

Table 3: Cross-workload comparison on CAPRI1 (rv32ic, 20 MHz). Cycles from gate-level simulation with SDF timing; E1 additionally validated on fabricated silicon. Faults: misclassifications from single-bit SRAM injection.

	Cycles			Faults			
Format	E1/2	E3	E4	E1	E2	E3	E4
LOG8-MAX	**1,622**	**2,615**	**17,695**	**0**	12	5	0
LOG8-SUM	3,985	8,028	39,017	3	20	6	8
MXINT8	4,256	10,854	36,612	3	10	**0**	**1**
MXFP8-E5M2	5,407	11,850	32,207	6	21	20	24
MXFP8-E4M3	5,511	12,122	34,603	25	0	22	5

E1: Self-Attention (28 wt, /280). E2: Mixture-of-Experts Attention (28 wt, /280).
E3: Multi-Layer Perceptron (114 p, /912). E4: Keyword-Spotting Attention (106 p, /848).
E1/2: same weight count; cycles shown once.

Table 4: Misclassification vulnerability across five MX formats (Exp 1). Each format undergoes 280 single-bit SRAM faults (35 bytes × 8 bits).

Rank	Format	Misclass.	Rate (%)	Dominant mechanism
1	**LOG8-MAX**	**0/280**	**0.0%**	Self-healing
2	MXINT8	3/280	1.1%	MSB-only
2	LOG8-SUM	3/280	1.1%	W_V mid-bit
4	MXFP8-E5M2	6/280	2.1%	W_V/W_O
5	MXFP8-E4M3	25/280	**8.9%**	Routing inversion

distance tracks fault propagation from initial bit flip to final state, capturing intermediate disturbance and self-healing (Table 8).

Of the 280 faults per format on the primary workload, 56 target shared-exponent bytes (7 SE × 8 bits) and 224 target element bytes (28 elements × 8 bits). This decomposition enables direct measurement of SE amplification (Table 6). Energy estimates use 180 nm operation costs from [11] (Table 1) and are not measured on chip. Each campaign uses one fixed input per workload; reported counts are conditional on that input's classification margin and should be read as format-comparative under matched conditions, not as input-marginal rates.

6 Results

We summarize fault counts across workloads (Section 6.1), rank formats on the primary workload (Section 6.2), and trace the disparity to three mechanisms (Sections 6.3–6.5).

6.1 Fault Attack Summary on Workloads

Table 3 summarizes fault counts across all four workloads. LOG8-MAX and MXINT8 achieve the lowest misclassification counts on three of four workloads. Vulnerability rankings vary by workload: MXFP8-E4M3 is most vulnerable on the primary workload (25/280) but has zero misclassifications on E2. This flip in ranking tracks which arithmetic stage decides the output: E1's softmax routing through W_O exposes MXFP8-E4M3 exponent-shift inversions, while E2's MoE max-gate is sensitive to LOG8-MAX (12/280) and absorbs the same MXFP8-E4M3 perturbations on the correct expert (0/280). The following subsections analyze the primary workload (Exp 1: Tiny Attention) in detail.

6.2 Vulnerability Ranking

Table 4 ranks all five formats on the primary workload. MXFP8-E4M3 is 8× more vulnerable than MXINT8 (8.9% vs. 1.1%), while LOG8-MAX achieves zero misclassifications. The two floating-point formats account for 31 of 37 total misclassifications (83.8%); MXINT8 and LOG8-SUM each contribute 3. Table 5 breaks this down by weight matrix and confirms that the ranking holds across all four projection matrices.

Table 5: Vulnerability heatmap: misclassification count by format and weight matrix (Exp 1). MXFP8-E4M3's W_O (9/40 fault sites) is the most fragile; LOG8-MAX is uniformly zero.

	SE	W_Q	W_K	W_V	W_O
LOG8-MAX	**0**	**0**	**0**	**0**	**0**
MXINT8	1	**0**	**0**	**0**	2
LOG8-SUM	**0**	**0**	**0**	3	**0**
MXFP8-E5M2	2	**0**	**0**	3	1
MXFP8-E4M3	6	3	5	2	9

0 1–2 3–5 6–8 ≥9

Table 6: Shared-exponent (SE) amplification. SE: 7 bytes × 8 bits = 56 faults. Element: 28 bytes × 8 bits = 224 faults.

Format	SE misclass.	SE rate	Elem misclass.	Elem rate	Amplif.
MXINT8	1/56	1.8%	2/224	0.9%	**2.0×**
MXFP8-E5M2	2/56	3.6%	4/224	1.8%	**2.0×**
MXFP8-E4M3	6/56	10.7%	19/224	8.5%	**1.3×**
LOG8-SUM	0/56	0.0%	3/224	1.3%	0
LOG8-MAX	0/56	0.0%	0/224	0.0%	N/A

Table 7: Corruption type classification. Severe routing: attention shift ≥253/256. Only MXFP8-E4M3 exhibits routing inversion.

Format	Severe rout.	Mild rout.	Value corr.	Output corr.	Total
MXINT8	0	0	0	2	3†
MXFP8-E5M2	0	0	3	3	6
MXFP8-E4M3	**8**	1	2	14	25
LOG8-SUM	0	0	3	0	3
LOG8-MAX	0	0	0	0	0

† 1 MXINT8 fault causes misclass. without detectable intermediate corruption.

6.3 Shared-Exponent Amplification

Table 4 identifies *which* formats are vulnerable; the next three subsections trace *why*. Table 6 separates misclassifications into SE faults (56 per format) and element faults (224 per format). MXINT8 and MXFP8-E5M2 both exhibit 2.0× SE amplification: per byte, an SE fault is twice as likely to cause misclassification as an element fault. MXFP8-E4M3 has lower amplification (1.3×) because its element faults are also highly damaging due to the two-level exponent. On Exp 1, LOG8-SUM and LOG8-MAX produce zero SE misclassifications; the log encoding bounds SE-induced magnitude shifts.

6.4 Attention Routing Inversion

The second mechanism targets the attention dataflow in Figure 3. Table 7 classifies each misclassification by corruption type. MXFP8-E4M3 alone exhibits **severe routing inversion**: eight faults in W_K (5) and W_Q (3) invert the 2×2 attention matrix from $\left[\begin{smallmatrix} 1 & 254 \\ 254 & 1 \end{smallmatrix}\right]$ to $\left[\begin{smallmatrix} 254 & 1 \\ 1 & 254 \end{smallmatrix}\right]$ (out of 256), routing Token 0's information to Token 1's position. MXFP8-E4M3's 4-bit element exponent shifts a decoded key magnitude by 2× to 64× per bit flip, sufficient to invert the softmax output. No other format produces this failure: MXINT8 lacks an element exponent, MXFP8-E5M2's wider range absorbs single-bit shifts, and the log encoding in LOG8-SUM/LOG8-MAX bounds error magnitude.

W_O is the most fragile matrix in MXFP8-E4M3: 14 of 25 misclassifications (56%) originate from its 40 fault sites (35% per-bit vulnerability). W_O maps context directly to logits without softmax normalization, so errors pass to the classifier without attenuation.

6.5 Log-Domain Self-Healing

The third mechanism is specific to LOG8-MAX. Table 8 lists fault propagation metrics for all five formats. In LOG8-MAX, **88 of 280 faulted runs (31.4%) converge back to the exact golden state**

Table 8: Fault propagation summary (mean across 280 faults per format). Self-heal = faults where final HD returns to zero.

Format	Peak HD	Final HD	Affected FFs	Self-healed	Misclass.
MXINT8	125	90	303	0	3
LOG8-SUM	148	105	370	0	3
LOG8-MAX	182	118	431	**88**	**0**
MXFP8-E5M2	288	232	737	0	6
MXFP8-E4M3	313	256	782	0	25

Table 9: Predicted vs. measured fault containment (Δz_j=0), Exp 1.

Format	Predicted	Measured	Δ
MXINT8	0.125^a	0.130	0.005
MXFP8-E4M3	$\sim 0.50^b$	0.506	0.006
MXFP8-E5M2	$\sim 0.30^b$	0.323	0.023
LOG8-SUM	0.125^a	0.132	0.007
LOG8-MAX	$\geq 0.971^c$	**0.982**	0.011

^{a}Zero-weight fraction k_0/n. ^{b}Includes NaN-clamping. ^{c}Property 1.

by program completion, despite initial pipeline disturbance. This *self-healing* differs from error masking: the internal state returns to the exact golden bit pattern, confirmed by zero Hamming distance at the final scan-chain capture.

Two mechanisms reinforce self-healing. Log-domain multiplication (addition of 7-bit integers) bounds the worst-case perturbation to $2^{0.5} \approx 1.41\times$ per fractional-bit flip. Integer softmax normalization (sum to 256) then absorbs these bounded errors during attention-row renormalization. Together, bounded perturbation and softmax absorption produce convergence despite substantial intermediate disturbance (mean peak HD = 182). LOG8-SUM shares the same encoding but accumulates through summation, so every faulted term shifts the output and no self-healing occurs. MXFP8-E4M3 has the worst propagation (mean peak HD = 313, 782 affected FFs) because its two-level exponent creates unbounded error growth that softmax cannot absorb. Table 9 validates the predicted containment bounds against measurement (deviations within 2.3% across all five formats).

7 Discussion and Related Work

Format choice as a security parameter. MX element encoding alone produces an 8× vulnerability gap under identical faults (Table 4), tracking each format's decode arithmetic: linear (MXINT8), exponential (MXFP8-E4M3), or bounded log-domain (LOG8-SUM/LOG8-MAX). The 95% Clopper–Pearson bound on the 0/280 LOG8-MAX rate is 1.3%; containment and self-healing are structural max-reduction properties, so accuracy on larger models needs separate validation. **Trade-offs.** LOG8-MAX leads in speed (2–4.6×) and energy but caps golden accuracy at 50% on multi-token attention; MXINT8 leads in resilience (0.12% on Exp 4) at full accuracy; LOG8-SUM is multiply-free (∼5 cyc/MAC) and full-accuracy at a 0.66%–7.1% fault rate. MXFP8-E4M3 is widely deployed yet most vulnerable; mitigations include LOG8-MAX where max-reduction is acceptable, targeted hardening of the softmax-bypassing W_O projection, and attention weight-distribution monitoring. Breier et al. [15] flags INT8 as the most resilient standard format under EMFI; we extend that finding to MX with shared-exponent amplification as an additional fault vector.

MX hardware and DNN fault attacks. MXDOTP [8], VMXDOTP [9], and Cuyckens et al. [10] add MX dot-product instructions or precision-scalable MX hardware to RISC-V cores but do not study fault behavior. Prior DNN fault-attack work [2–4] targets FP32 weights via bit-flip, rowhammer, and random upset; Breier [5] attacks fixed-point accelerators and [15] reports that standard floating-point formats collapse under EMFI while integer ones stay functional; none examine MX block-scaled formats.

Table 10: This work vs. prior DNN fault-injection studies. MX = microscaling; SE = shared exponent.

Work	Formats	MX/SE	Platform	Faults
[2–4, 14]	FP32	×	Sim/DRAM	Bit-flip/RH
[5]	INT8	×	FPGA	Laser/EMFI
[15]	4 std	×	SRAM	EMFI
This work	**5 MX**	✓	**ASIC+Sim**	**Exhaustive**

Mitigations and log-number systems. TMR, ABFT, and Razor protect at area or latency cost, and Unicorn-CIM [12] adds ECC at 8.98% overhead; LNS-Madam [13] substitutes log-domain adders for multipliers but retains linear-domain summation, and Dally's LOG8 encoding [7] (adopted here) is extended by LOG8-MAX, which removes the conversion bottleneck via max-reduction and obtains fault filtering as a structural byproduct at zero hardware cost. Table 10 positions this study against prior work.

8 Conclusion

Arithmetic structure, not bit width alone, is a security parameter in MX accelerator design. Across five MX-format encodings on a RISC-V SoC and four workloads (the primary one silicon-validated), three MX-specific mechanisms (shared-exponent amplification, attention routing inversion, log-domain fault filtering) drive a format-dependent disparity in which no format dominates all axes.

Acknowledgments

This research was supported in part by the National Science Foundation under Grant No. 2219810.

References

[1] B. Darvish Rouhani et al. OCP Microscaling formats (MX) specification, v1.0. Open Compute Project, 2023.
[2] A. S. Rakin et al. Bit-flip attack: Crushing neural network with progressive bit search. *Proc. ICCV*, 2019, 1211–1220. https://doi.org/10.1109/ICCV.2019.00130.
[3] F. Yao et al. DeepHammer: Depleting the intelligence of deep neural networks through targeted chain of bit flips. *USENIX Security*, 2020, 1463–1480.
[4] S. Hong et al. Terminal brain damage: Exposing graceless degradation in DNNs under hardware fault attacks. *USENIX Security*, 2019, 497–514.
[5] J. Breier et al. Practical fault attack on deep neural networks. *ACM CCS*, 2018, 2204–2206. https://doi.org/10.1145/3243734.3278519.
[6] A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017.
[7] W. J. Dally. Trends in deep learning hardware: Log number systems for energy-efficient inference. Stanford SystemX, 2024.
[8] G. Islamoglu et al. MXDOTP: A RISC-V ISA extension for microscaling dot products. *IEEE ASAP*, 2025, 81–84.
[9] M. Wipfli et al. VMXDOTP: A RISC-V vector ISA extension for MX format acceleration. *DATE*, 2026.
[10] S. Cuyckens et al. Efficient precision-scalable hardware for microscaling processing in robotics learning. *ISLPED*, 2025.
[11] M. Horowitz. Computing's energy problem (and what we can do about it). *ISSCC*, 2014, 10–14. https://doi.org/10.1109/ISSCC.2014.6757323.
[12] Q. Li, Y. Liang, and W. Cao. Unicorn-CIM: Uncovering vulnerability and improving resilience of high-precision compute-in-memory. *ISLPED*, 2025, 1–6.
[13] J. Zhao et al. LNS-Madam: Low-precision training in logarithmic number system. *IEEE Trans. Comput.*, 71(12):3179–3190, 2022.
[14] B. Reagen et al. Ares: A framework for quantifying the resilience of deep neural networks. *DAC*, 2018, 1–6. https://doi.org/10.1109/DAC.2018.8465834.
[15] J. Breier, Š. Kučerák, and X. Hou. The weight of a bit: EMFI sensitivity analysis of embedded deep learning models. *arXiv:2602.16309*, 2026.
[16] G. Agosta, A. Galimberti, and D. Zoni. Deep learning on RISC-V platforms at the edge. *ACM Comput. Surv.*, 58(5):art. 121, 2025.
[17] D. Shanmugam et al. Hierarchical EMFI analysis on a RISC-V SoC. *IEEE ETS*, 2026 (accepted).